Kolkata Metro Management System

End-to-end implementation walkthrough, submission notes, and interview preparation

Assessment stack: React + Vite, FastAPI, PostgreSQL, SQLite

Verification code: `HI-U-PASSED-THE-DEV-CHALLENGE`

Live frontend: `https://frontend-production-9fb5.up.railway.app`

Live backend: `https://backend-production-15ec.up.railway.app`

GitHub repository: `https://github.com/vaibwork/kolkata-metro-ticket-booking-app.git`

APK artifact: `release-apk/kolkata-metro-planner-live.apk`

Current date: 17 July 2026

Latest known commit: `b7711a3 Organize submission screenshots`

1. Original Assignment in Plain Terms

The project started as a partially completed fictional Kolkata Metro ticket booking application. The task was not just to write new features, but to understand an unfamiliar full-stack codebase, make it run, complete missing backend and frontend functionality, and submit proof that the system works.

Requirement	Meaning	Delivered result
Run the project	Install dependencies, fix setup blockers, start frontend and backend.	Local and Railway deployments were made functional.
Show verification code	Homepage had to prove PostgreSQL, SQLite, and heartbeat checks pass.	Code shown: `HI-U-PASSED-THE-DEV-CHALLENGE`.
Display all stations	Use the existing backend API and show station data in the frontend.	Frontend station list integrated with loading and error states.
Shortest route finder	Complete missing backend logic inside the existing endpoint, using SQLite.	Dijkstra graph engine implemented without changing database schema.
Integrate route finder	Let users select source and destination, then show route output.	Dropdown selection, required demo routes, route summary, full route, and leg breakdown added.
Submit assets	Verification code, screenshots, public repo URL, walkthrough video, and TESTIMONIAL.md.	Submission screenshots are under `submission-assets/`; TESTIMONIAL.md exists in root.

2. Final Delivered System

The final application is a working full-stack metro route planner and QR ticket workflow. It displays stations from SQLite, calculates shortest routes from SQLite graph data, stores booked tickets in PostgreSQL, generates scannable QR codes, and exposes a validation endpoint that could be used later by a gate scanner.

Station browser · Shortest route finder · Fare estimate · Travel time · Interchange count · Leg breakdown · PostgreSQL ticket registry · Real QR payload · Gate validation API · Dark/light mode · Railway deployment · Capacitor APK

3. Chronological Work Sequence

1. **Read the assessment brief.** The important constraints were identified first: no schema changes, no station data changes, no new route endpoint, preserve the API contract, and use SQLite for route calculation.
2. **Cloned and inspected the repository.** The codebase was separated into backend, frontend, and database setup areas. The SQLite DDL documentation was treated as the source of truth for graph structure.
3. **Fixed setup issues.** Missing dependencies and path/config problems were handled so the backend and frontend could start. Frontend needed dependencies such as `lucide-react`; backend needed SQLAlchemy-related setup.
4. **Fixed SQLite access.** The backend was adjusted to use the configured SQLite path relative to the repository instead of looking in the wrong module path.
5. **Integrated station listing.** The frontend calls the existing station API, renders the station list, and handles loading/error states.
6. **Implemented route graph logic.** SQLite stations, connections, and interchanges are loaded into a weighted graph. Dijkstra is used to find the shortest path.
7. **Handled duplicate station names.** Some stations can exist across lines. The backend supports station IDs while keeping source/destination names valid, so the API remains backward compatible.
8. **Connected route planner UI.** Users can select source/destination, use one-click required assessment routes, see summary metrics, then expand the full itinerary and leg breakdown.
9. **Built ticket booking flow.** Route results can be booked as QR tickets, stored in PostgreSQL, and selected in the registry.
10. **Added QR validation path.** QR payloads can be scanned by a normal QR scanner to reveal validation information, and a backend validation endpoint can later be used by a gate system.
11. **Improved UX.** Diagnostics were made less intrusive, route details were collapsed by default, booking registry layout was improved, QR ticket was moved above registry on narrower layouts, and dark/light mode plus motion polish was added.

12. **Deployed on Railway.** Backend, frontend, and PostgreSQL were deployed. The frontend deployment required Docker/static server fixes to avoid Railway build/start issues.
13. **Built Android APK.** Capacitor was added as a native shell that loads the live Railway web app without a browser address bar.
14. **Captured submission screenshots.** Required route screenshots were generated and organized under `submission-assets/`.
15. **Prepared repository for submission.** README, TESTIMONIAL.md, screenshots, and APK artifact were pushed to the public repository.

---

4. Architecture

Layer	Responsibility	Main reason
React + Vite frontend	User interface, station selection, route display, ticket registry, QR display, theme, responsive layout.	Fast development and clear component structure.
FastAPI backend	REST API, route calculation, station reads, ticket operations, status diagnostics, QR validation.	Clean API layer with Python data processing.
SQLite	Static metro graph: stations, connections, interchanges, verification vault data.	Assessment explicitly required using the provided SQLite database and preserving station data.
PostgreSQL	Runtime system data: booked tickets, configuration/verification, heartbeat-backed status.	Better for server-side operational records than editing the static SQLite dataset.
Railway	Production hosting for frontend, backend, and PostgreSQL.	Single platform for public submission and live verification.
Capacitor Android	Native APK wrapper around the live web app.	Provides app-like mobile experience without an address bar and receives live web updates.

Data flow: User selects stations in React -> frontend calls FastAPI -> FastAPI reads SQLite graph -> Dijkstra calculates route -> frontend shows summary/details -> user books ticket -> FastAPI writes ticket to PostgreSQL -> frontend displays QR -> scanner can call validation endpoint.

---

5. Backend Route Logic

The core missing backend feature was completed in `backend/app/services/graph_engine.py`. The implementation reads the existing SQLite schema and builds a graph where each station-line record is a node. Track connections and interchange links become weighted edges.

Concept	Implementation detail
Nodes	Station rows from SQLite. A visible station name can map to more than one internal station ID.
Train edges	Rows from the connections table. Weights use travel time and fare.
Interchange edges	Rows from interchanges. These allow changing lines, usually with transfer time and zero fare.
Algorithm	Dijkstra with a priority queue because all edge weights are non-negative.
Tie behavior	Routes are compared by time first and fare second, using a tuple-style cost.
Path output	The previous-node map reconstructs the ordered itinerary and route legs.

> **Why Dijkstra:** BFS only minimizes number of edges, not travel time or fare. Since metro segments can have different durations and interchange costs, weighted shortest path is the correct model.

The route endpoint remained the existing route endpoint. Optional station IDs were added for precision, while the original source/destination name inputs still work. That preserves the API contract while solving duplicate-name ambiguity.

---

6. Frontend Route Planner

The frontend was upgraded from a basic/incomplete state into a working route planner. The user can choose source and destination from station data returned by the API. The app prevents invalid same-station selections and handles API failure states.

UI area	Behavior
Station selectors	Loaded from API and used to pass source/destination to the route endpoint.
Assessment routes	One-click buttons for Dakshineswar -> VIP Bazar, Park Street -> Howrah, and Thakurpukur -> Eco Park.
Route summary	Shows fare, travel time, and interchanges first so the user can book quickly.
Full route	Collapsed by default to reduce scrolling; can be expanded for screenshot and review.
Leg breakdown	Collapsed by default; explains line-by-line travel and interchange sections.
Book button	Creates a ticket in PostgreSQL and auto-selects the new ticket in the QR panel.

---

7. Ticketing and QR Workflow

The ticket flow makes the assessment feel closer to a real transit app while staying within the project scope. When a user books a route, the backend creates a ticket number like `KMETRO-XXXXXXXX`, stores fare, source, destination, expiry, and status in PostgreSQL, then returns it to the frontend.

The QR code is generated from a real payload, not a decorative image. A normal QR scanner can read it and show ticket/validation information. A future gate device could scan the QR, extract the ticket number, and call the backend validation endpoint.

State	Validation behavior
Active and not expired	Gate-style response can return access granted.
Expired	Access denied with reason.
Unknown ticket	Access denied because ticket does not exist.

8. UI and UX Improvements Beyond the Minimum

The minimum assessment only required stations and route display. Additional improvements were added carefully to make the submission more polished without violating the original API/database constraints.

- **Dark/light mode:** Shows design maturity and improves accessibility across environments.
- **Motion effects:** Adds professional feedback without distracting from the workflow.
- **Compact diagnostics:** Verification details were moved away from the main interaction area so the homepage looks like an app, not a debug page.
- **Collapsed route details:** Fare, time, and interchanges are visible first; full route and leg breakdown expand only when needed.
- **Booking registry redesign:** Avoids horizontal scrolling/clipping on desktop and uses more suitable stacked behavior on smaller layouts.
- **Auto-select new ticket:** After booking, the QR appears immediately, reducing user confusion.
- **QR above registry on mobile/narrow screens:** The most important post-booking action is visible sooner.
- **Capacitor APK:** Demonstrates future mobile direction with an app-like shell.

9. Railway Deployment Issues and Fixes

Deployment was part of making the work submission-ready. The backend and database were connected to Railway PostgreSQL. The frontend needed extra care because Railway's automatic detection had build/start problems.

Issue	Cause	Fix
Frontend build/start failed	Railway/Railpack detected the wrong root or lacked the expected npm environment in that build context.	Added a deterministic frontend Dockerfile and deployed with the frontend path as root.
Frontend called localhost in production	Vite injects environment variables at build time, not runtime.	Set `VITE_API_URL=https://backend-production-15ec.up.railway.app/api` during Docker build.
Frontend produced 502	Vite preview and Railway start command/port handling were not reliable enough for this setup.	Added `frontend/server.js`, a Node static server that binds to `process.env.PORT`.

Final production links: frontend `https://frontend-production-9fb5.up.railway.app`, backend `https://backend-production-15ec.up.railway.app`.

10. Android APK

Capacitor was used to create an Android shell for the web app. The app opens without a browser address bar and points at the live Railway frontend. This means normal web deployment updates appear in the installed APK without rebuilding the APK.

> **Important production note:** This is good for a demo/internal assessment APK. A real app-store production release would need a signed release key, versioning strategy, privacy/security review, and careful policy handling for remote web content.

APK path in repo: `release-apk/kolkata-metro-planner-live.apk`. GitHub browser link: `https://github.com/vaibwork/kolkata-metro-ticket-booking-app/blob/master/release-apk/kolkata-metro-planner-live.apk`.

11. Verification and Testing

Check	Result
Backend tests	`9 passed` with pytest.
Frontend build	`npm run build` passed after fixes.
Live frontend	HTTP 200 from Railway frontend.
Live backend health	HTTP 200 from `/api/health`.
Live status	`/api/status` returns successful diagnostics and verification code when services are healthy.
Station API	`/api/allstations` returns station data.
Tickets API	`/api/tickets` returns ticket registry data.

12. Submission Checklist

Google Form field	What to submit
Email, name, college, year	Your personal details.
Verification code	`HI-U-PASSED-THE-DEV-CHALLENGE`
Repository URL	`https://github.com/vaibwork/kolkata-metro-ticket-booking-app.git`
Homepage screenshot	`submission-assets/homepage-screenshot.png`
Route screenshots	`screenshot-dakshineswar-vip-bazar.png`, `screenshot-park-street-howrah.png`, `screenshot-thakurpukur-eco-park.png`
Walkthrough video	Record the live app: verification, station list, each required route, booking QR, registry, theme/mobile responsiveness.
How did you resolve bugs and execute code?	Use the answer in the next section, adjusted to your voice.

13. Suggested Form Answer

I first reviewed the project structure, database documentation, and existing API contracts so I could fix the application without changing the intended architecture. During setup I resolved dependency/configuration problems, corrected the SQLite database path, configured the backend/frontend API connection, and verified that FastAPI, React/Vite, PostgreSQL, and SQLite were all working together.

For the station feature, I integrated the existing backend station API with the frontend and added loading/error handling. For the shortest route feature, I completed the missing backend logic inside the existing route endpoint. I used the SQLite station, connection, and interchange data to build a weighted graph and applied Dijkstra's algorithm to compute the shortest route. I kept the existing API contract and database schema unchanged, while supporting station IDs internally to avoid ambiguity where station names appear on multiple lines.

After the route calculation worked, I integrated it into the frontend with source/destination selectors, route summary, fare, travel time, interchanges, full route details, and leg breakdown. I also improved the UX beyond the minimum requirement by adding dark/light mode, better responsive layout, collapsible route details, one-click assessment routes, QR ticket booking, ticket registry, and a QR validation flow that can support future gate-scanner behavior. I deployed the backend, frontend, and PostgreSQL database on Railway, fixed production build/runtime issues using a deterministic Docker setup and Node static server, added a Capacitor Android APK shell, and verified the app with tests, builds, live health checks, and required route screenshots.

14. Walkthrough Video Script

1. Open the live app and mention that this is a fictional assessment project, not an official metro system.
2. Show the verification/status area and read the clearance code.
3. Show the station list loaded from the backend API.
4. Use one-click demo route Dakshineswar -> VIP Bazar; expand full route and leg breakdown.
5. Repeat for Park Street -> Howrah and Thakurpukur -> Eco Park.
6. Book one route and show that the new ticket auto-selects.
7. Show the QR ticket and explain that the QR contains a scannable ticket validation payload.
8. Open the booking registry and show ticket status/fare/route.
9. Toggle dark/light mode and resize/navigate on mobile if recording supports it.
10. End by showing the GitHub repository, TESTIMONIAL.md, screenshots folder, and APK artifact.

15. Interview Questions and Strong Answers

Q1. What was the main task in this assessment?

To understand a partial full-stack app, fix setup issues, integrate all-stations display, implement shortest route calculation inside the existing backend endpoint using SQLite, integrate it into React, and provide submission proof through screenshots, repository, video, and TESTIMONIAL.md.

Q2. Why did you read the SQLite DDL first?

Because the route logic depends on the actual schema. I needed to know how stations, connections, and interchanges were represented before building the graph. Guessing the schema would risk wrong joins or incorrect routes.

Q3. Why did you use Dijkstra's algorithm?

The graph has weighted edges such as travel time and interchange cost. Dijkstra is appropriate because weights are non-negative and we need the minimum weighted path, not just the fewest stops.

Q4. Why not use BFS?

BFS minimizes number of edges. A route with fewer stops may still take longer or cost more if segment weights differ. The assessment asked for shortest route using the database, so weighted path logic is more correct.

Q5. How did you model interchanges?

Interchanges are graph edges between station-line nodes. They allow the route engine to move from one line representation of a station to another while applying transfer time and usually zero fare.

Q6. How did you handle duplicate station names?

A station name can map to multiple station IDs on different lines. The route engine supports multi-source/multi-destination matching and the frontend can pass IDs for precise selection, while still keeping the original source/destination name contract valid.

Q7. Did you change the API contract?

No. The existing route endpoint remains the route endpoint and still accepts the expected source and destination inputs. Optional IDs only improve precision and do not break existing callers.

Q8. Did you modify station data or database schema?

No. The station data and schema were preserved. The implementation reads the existing SQLite data and builds runtime graph structures in memory.

Q9. What were the main bugs during setup?

The important setup issues were missing dependencies, incorrect SQLite path/config expectations, frontend API base URL issues, CORS/local development configuration, and later Railway frontend build/runtime issues.

Q10. Why use SQLite and PostgreSQL both?

SQLite is used for the static assessment dataset and route graph. PostgreSQL is used for operational data such as tickets, system configuration, and heartbeat/status checks. This separation avoids mutating the provided station dataset.

Q11. How does the ticket QR work?

A booked ticket is stored in PostgreSQL and rendered as a QR payload. A scanner can read the payload, and a future gate can validate the ticket number through the backend validation endpoint.

Q12. Is the QR production secure?

It is a strong demo foundation, but production would require signed/encrypted QR payloads, anti-replay protection, audit logs, rate limiting, and stricter expiration/device validation.

Q13. Why did you add a validation endpoint?

It makes the QR workflow realistic. The QR is not only a visual artifact; it connects to a backend path that can decide whether gate access should be granted or denied.

Q14. Why did you collapse full route and leg breakdown?

The user first needs fare, time, interchanges, and the book button. Full route and leg details are useful but can create too much scrolling, so they are available on demand.

Q15. Why put QR above registry on mobile?

After booking, the most important output is the ticket/QR. On mobile, putting QR above the registry reduces scrolling and matches the real-life ticket-checking workflow.

Q16. Why add dark/light mode?
It was not required, but it demonstrates UX thinking, accessibility awareness, and a more complete app feel without changing backend requirements.

Q17. How did you make the app responsive?
The layout uses adaptive sections and reorders important panels for narrower screens. Dense tabular data is simplified so users are not forced to horizontally scroll for key ticket information.

Q18. What tests did you run?
I ran backend pytest tests, frontend production builds, and live endpoint checks for frontend, backend health, status, stations, and tickets. Required screenshot routes were also verified visually.

Q19. What did Railway deployment teach you?
Deployment exposed differences between local and production environments. Vite needs API URLs at build time, and a deterministic Docker/static-server setup was more reliable than relying on automatic detection.

Q20. Why was VITE_API_URL important?
Vite embeds environment variables into the built JavaScript. If the production build is created without the Railway backend URL, the deployed frontend can accidentally call localhost.

Q21. Why use a Node static server?
It makes the frontend startup predictable on Railway. It serves the built Vite assets and binds to the port Railway provides through `process.env.PORT`.

Q22. What is the purpose of the APK?
The APK demonstrates a real mobile app direction. It opens without a browser address bar and loads the live web app, so updates deployed to Railway appear in the installed app.

Q23. What are limitations of the APK approach?
It depends on network access and the live web app. For production, I would add offline handling, signed release builds, secure update policy, native splash/icon polish, and store-compliant configuration.

Q24. What would you improve with more time?
Add authenticated admin tools, stronger QR security, route caching, more tests around tricky interchange routes, observability, CI/CD, accessibility audit, and a polished signed mobile release pipeline.

Q25. How would you explain your engineering approach?
I first respected the constraints, then made the system run, then implemented the missing core logic, then improved reliability and UX, and finally packaged the submission with proof artifacts.

Q26. How do you know the result follows the assignment?
The route endpoint was not replaced, station data/schema were not modified, SQLite is used for route calculation, the frontend displays stations and routes, and the required screenshots/repository/testimonial assets are present.

Q27. How would you debug a wrong route?
I would inspect the matching source/destination IDs, print graph neighbors for those nodes, verify connection and interchange weights from SQLite, then compare the reconstructed path with expected line behavior.

Q28. How would you scale the route engine?
For this dataset, in-memory Dijkstra is enough. At larger scale, I would cache the graph, precompute common paths, use better graph libraries if needed, and add performance metrics around route queries.

Q29. How would you secure the backend?
Add authentication for privileged operations, rate limiting, input validation, stricter CORS, secret management, signed QR tokens, HTTPS-only deployment, and structured audit logs.

Q30. What part best shows problem-solving?
The route engine and deployment fixes. The route engine required understanding schema and graph modeling; the deployment required solving build-time environment and runtime hosting differences.

---

16. Key Things to Remember Before Interview

- Say clearly that this is a fictional assessment project and not connected to a real transit authority.
- Emphasize constraints: no schema changes, no station data modifications, no new route endpoint.
- Explain Dijkstra simply: weighted graph, non-negative costs, priority queue, previous map, reconstructed path.
- Explain SQLite vs PostgreSQL separation: static route dataset vs dynamic ticket/system records.
- Be honest that the QR is demo-ready, while production would need signed payloads and stronger anti-fraud controls.
- When asked about UI, say the goal was to make the primary workflow fast: select route, see summary, book ticket, show QR.
- When asked about deployment, mention Vite build-time env variables and Railway port handling.
- When asked what you learned, talk about reading existing code first, preserving contracts, and validating locally plus in production.

---

17. Final Submission Links and Files

Item	Value
Repository	`https://github.com/vaibwork/kolkata-metro-ticket-booking-app.git`
Live frontend	`https://frontend-production-9fb5.up.railway.app`
Live backend	`https://backend-production-15ec.up.railway.app`
Verification code	`HI-U-PASSED-THE-DEV-CHALLENGE`
Homepage screenshot	`submission-assets/homepage-screenshot.png`
Route screenshots	`submission-assets/screenshot-dakshineswar-vip-bazar.png`, `submission-assets/screenshot-park-street-howrah.png`, `submission-assets/screenshot-thakurpukur-eco-park.png`
APK	`release-apk/kolkata-metro-planner-live.apk`
Testimonial	`TESTIMONIAL.md`